

对 Team 1 模型的分析报告

Team 3: Liang Dongcheng, Zhang Lingwei, Jin Peiyao

2025.11.18

在初步修改本文档的一些文件读取、版本兼容等问题后，我们能够复现这份代码的结果。这份代码的优点在于能够复现出具体结果（如下各个 Cell 所示），且数值处理较好，细节处理优秀。

	Model	Train MAE (Log)	Test MAE (Log)	CV MAE (Log)	\
0	OLS	0.26	0.26	0.26	
1	Ridge	0.26	0.26	0.26	
2	Lasso	0.27	0.27	0.27	
3	ElasticNet	0.28	0.28	0.28	

	Train MAE (Original)	Test MAE (Original)	CV MAE (Original)
0	168,491.35	162,921.47	168,579.47
1	168,493.16	162,923.97	168,581.67
2	172,113.15	166,749.94	172,187.51
3	181,029.71	175,677.96	181,101.83

[43]: ['scaler_rent.pkl']

但是，在具体处理结果的时候，有一些具体的小问题，这里给出一点小建议。

Cell 2、8

当前代码中对 `train_price / train_rent` 的列删除非常激进，不少明显对价格有影响的变量被直接删掉，比如：房屋朝向（采光、通风都会影响成交价和租金）；梯户比例、套内面积（反映居住舒适度、使用面积）等。建议房屋朝向可以做成 One-Hot 或者简单分两类（南 / 非南），梯户比例和套内面积可以根据缺失量大小酌情考虑。同理，在对 `train_rent` 的 Cell 8 中也是如此建议。

```
train_price = housing['train_price'].copy()
train_price = train_price[train_price['别墅类型'].isna()]
train_price.drop(columns=['梯户比例', '套内面积', '楼栋位置', '房屋朝向', '别墅类型', '交易时间', '交易权属', '上次交易', '房屋用途', '产权所属', '抵押信息', '房屋优势', '核心卖点', '户型介绍'], inplace=True)
```

Cell 7

Cell 7 中得到的具体结果如下：对 `train_price` 的多个数值特征采用 IQR 方法进行异常值替换，最后得到的所有值标准差为 0，但源数据得到的结果并不是这样子的。这说明在这个结果中所有特征的全部信息被丢失掉了，这可能没办法得到更准确的预测。可以考虑采用极值截断（Winsorizing）进行处理。

```
[7]: # 异常值 (IQR 方法)
for col in cols_to_fix:
    Q1 = train_price[col].quantile(0.25)
    Q3 = train_price[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    median_value = train_price[col].median()

    train_price[col] = train_price[col].mask((train_price[col] < lower_bound) | (train_price[col] > upper_bound), median_value)

train_price[cols_to_fix].describe()
```

另外，中位数填补和异常值处理可能导致数据泄漏：代码在“数据清洗阶段”直接使用全体数据计算中位数，用来填补缺失或替换异常值。为避免数据泄漏，缺失值填补和异常值处理的中位数 / 分位数应仅根据训练集计算，并通过 Pipeline 复用于验证 / 测试集，而不能在合并后的全数据上一次性计算。

[7]:

	绿化率 (%)	容积率 (倍)	物业费 (元/月/m²)	燃气费 (元/m³)	供热费 (元/m²)	停车位 (个)	停车费用 (元)
count	91520.000000	91520.000000	91520.000000	91520.000000	91520.0	91520.000000	91520.000000
mean	32.948525	2.449292	1.859527	2.631610	25.0	786.443477	310.471287
std	2.907591	0.635627	0.679447	0.456645	0.0	476.428844	31.540344
min	23.000000	0.790000	0.200000	1.730000	25.0	1.000000	230.000000
25%	30.200000	2.130000	1.475000	2.460000	25.0	500.000000	300.000000
50%	33.000000	2.500000	1.875000	2.610000	25.0	756.000000	300.000000
75%	35.000000	2.500000	2.075000	2.620000	25.0	885.000000	300.000000
max	42.300000	4.330000	3.810000	3.675000	25.0	2200.000000	420.000000

Cell 11

此外，Cell 11 当中有一个结果是处理 `train_rent`，但这里的代码被赋值为 `train_price`，这可能导致建模时候得到误差。建议修改为：

```
train_rent['绿化率 (%)'] = pd.to_numeric(train_rent['绿化率 (%)'], errors='coerce')
```

```
[11]: train_rent.rename(columns={
    '面积': '面积 (m²)',
    '绿化率': '绿化率 (%)',
    '容积率': '容积率 (倍)',
    '物业费': '物业费 (元/月/m²)',
    '燃气费': '燃气费 (元/m³)',
    '供热费': '供热费 (元/m²)',
    '停车位': '停车位 (个)'
}, inplace=True)

train_rent['绿化率 (%)'] = train_rent['绿化率 (%)'].astype(str).str.replace('%', '', regex=False)
train_rent['绿化率 (%)'] = pd.to_numeric(train_price['绿化率 (%)'], errors='coerce')
train_rent['面积 (m²)'] = train_rent['面积 (m²)'].astype(str).str.replace('m²', '', regex=False)
train_rent['燃气费 (元/m³)'] = train_rent['燃气费 (元/m³)'].astype(str).str.replace('元/m³', '', regex=False)
train_rent['供热费 (元/m²)'] = train_rent['供热费 (元/m²)'].astype(str).str.replace('元/m²', '', regex=False)
train_rent['物业费 (元/月/m²)'] = train_rent['物业费 (元/月/m²)'].astype(str).str.replace(r'[\d\.-]', '', regex=True)

train_rent.sample(5)
```

Cell 17

Cell 17 中将建筑结构编码强行赋予“顺序”，可能不合理。这里的 0-6 并没有真正的“单调顺序”，但对于线性模型来说，会被当成有序数值来用，可能带来错误的趋势假设。应当使用 One-Hot 编码。

```

# 生成映射字典
train_price['环线编码'] = train_price['环线编码'].fillna('未知编码', inplace=True)
structure_map = {
    '未知结构': 0,
    '普通结构': 1,
    '高级结构': 2,
    '豪华结构': 3,
    '顶级结构': 4,
    '超顶级结构': 5,
    '其他结构': 6,
}
train_price['建筑结构编码'] = train_price['建筑结构'].map(structure_map)
train_price[['环线编码', '建筑结构编码']].head(10)

```

Cell 29

Cell 29 中有一个问题是对环线编码、建筑结构编码进行了对数变换。这种变换虽能增强稳健性，但有一个问题是把数值特征急速缩小，会破坏编码的意义。可以考虑移除这里的对数变换。

```

•[29]: cols = ['Price', '建筑面积 (m²)', '环线编码', '建筑结构编码']
train_price['log_price'] = np.log1p(train_price['Price'])
train_price['log_area'] = np.log1p(train_price['建筑面积 (m²)'])
train_price['log_ringcode'] = np.log1p(train_price['环线编码']) #这一行对数变换可以考虑适当去除 (Team3)
train_price['log_stru_code'] = np.log1p(train_price['建筑结构编码']) #这一行对数变换可以考虑适当去除 (Team3)
for col in ['log_price', 'log_area', 'log_ringcode', 'log_stru_code']:
    print(col, '偏度 =', train_price[col].skew())

```

Cell 40

Cell 40 进行的是特征选择，标准化 (StandardScaler) 是在所有数据 X1 上操作，这意味着 X1test 的信息会泄露到 Lasso 特征选择过程中。这会使得选出的特征对模型的改进表现过于乐观，夸大了模型在未知数据上的泛化能力。

建议先分组做好样本划分 (train 和 test)，在 X1train 上进行 fit_transform，然后用这个拟合好的 Scaler 应用于 X1test 再进行 transform。

```

[40]: from sklearn.linear_model import LassoCV
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error

# 特征和目标
X1 = train_price.drop(columns=['log_price', 'Price'])
y1 = train_price['log_price']

# 标准化数据
scaler = StandardScaler()
X1_scaled = scaler.fit_transform(X1)

# 划分数据集
X1_train, X1_test, y1_train, y1_test = train_test_split(X1_scaled, y1, test_size=0.2, random_state=111)

# LassoCV 自动选择 alpha
lasso_cv = LassoCV(alphas=[0.01, 0.1, 1, 10, 100], cv=5) # 使用5折交叉验证
lasso_cv.fit(X1_train, y1_train)

# 打印最优的 alpha 和系数
print("最佳 alpha:", lasso_cv.alpha_)
print("LASSO 回归系数:", lasso_cv.coef_)

# 获取系数不为零的特征
selected_features = X1.columns[lasso_cv.coef_ != 0]
print("Selected features by LASSO:", selected_features)

# 在测试集上进行预测
y1_pred = lasso_cv.predict(X1_test)

# 计算 MAE (Mean Absolute Error)
mae = mean_absolute_error(y1_test, y1_pred)
print("Mean Absolute Error (MAE):", mae)

```